

Project Workflow & Dependency Guide

End-to-End ML Architecture · Build Phases · Tech Stack

Table of Contents

1. Project Overview & Vision	3
2. System Architecture Summary	4
3. Technology Stack & Dependencies	5-6
4. Phase 1 — Infrastructure & Data Layer Setup	7
5. Phase 2 — ML Feature Engineering Pipeline	8
6. Phase 3 — Model Development & Training	9
7. Phase 4 — Model Serving & API Layer	10
8. Phase 5 — Application & Frontend Build	11
9. Phase 6 — MLOps, Monitoring & CI/CD	12
10. End-to-End Data Flow	13
11. Risk Register & Mitigation	14
12. Timeline & Milestone Summary	15
App.. Quick-Reference Checklist	16-18

01 — PROJECT OVERVIEW & VISION

Project Overview & Vision

FinPulse is an AI-powered financial intelligence platform designed to help lenders and borrowers make smarter decisions using real-time ML-driven insights. The platform ingests raw banking and transaction data, processes it through a multi-stage feature pipeline, scores users across five predictive models, and surfaces results via dedicated dashboards and REST APIs.

Core Objectives

- Provide a real-time **Financial Health Score** (0-100) for every borrower.
- Predict **Loan Default Risk** with probabilistic output (Safe / Watch / High Risk).
- Detect **Transaction Anomalies** and potential fraud in real time.
- Forecast **Cash-Flow** over 7-30 day horizons to surface EMI failure risk.
- Generate personalised **Wellness Recommendations** to improve financial health.
- Serve all insights via secure **REST APIs** and role-based dashboards.

Stakeholders & Users

Stakeholder	Role	Primary Interface
Lenders / Banks	Consume risk scores for credit decisioning	Lender Risk Dashboard
Borrowers / Users	View health score & recommendations	Borrower Dashboard
Data Engineers	Manage ingestion & feature pipelines	Internal tooling / Airflow
ML Engineers	Train, evaluate & deploy models	MLflow / Model Registry
DevOps / Platform	Infrastructure, CI/CD, monitoring	Cloud console / Grafana

Design Principles

Principle	Description
Modularity	Each ML model is an independent microservice; upgrades to one do not affect others.
Feature Consistency	A dual offline/online feature store ensures training and inference use identical feature definitions.
Explainability	Gradient-boosted models and rule-based recommendation engines prioritise human-readable outputs.
Scalability	Containerised services behind load-balanced Prediction APIs scale horizontally on demand.
Observability	Every model endpoint is wrapped with latency tracking and accuracy-drift monitoring.

02 — SYSTEM ARCHITECTURE SUMMARY

System Architecture Summary

FinPulse follows a layered ML pipeline: **Data Sources** → **Data Processing** → **Feature Store** → **ML Models** → **Model Serving** → **Application** → **Frontend**. External services enrich raw data, and a dedicated database layer persists predictions and user data.

Architecture Layers

Layer	Components	Purpose
External Services	Credit Bureaus, Identity Verification, Banking APIs	Enrich borrower data with bureau scores and verified identity signals.
Data Sources	Bank Transactions, User Profile, Loan Info, Spending Categories, Balance History	Raw financial data ingested via the Data Ingestion API.
Data Processing	Data Ingestion API → Cleaning & Preprocessing → Feature Engineering → Aggregation	ETL pipeline that cleanses, standardises, and aggregates raw records into model-ready features.
Feature Store	Financial Features, Transaction Behaviour, User Metrics	Dual offline (batch / Spark) and online (real-time) stores ensuring training-inference consistency.
ML Models	Health Score, Loan Default Risk, Anomaly Detection, Cash-Flow Forecast, Wellness Recommendation	Five independent supervised, unsupervised, and time-series models.
Model Serving	Prediction APIs, ML Microservice, Model Registry, Monitoring	Containerised inference endpoints with a central registry and drift monitoring.
Application Layer	Django API, Authentication, Financial Reports	Secure backend handling auth, business logic, and reporting.
Database Layer	PostgreSQL — Transaction Data, Model Predictions, User Data	Persistent storage for all transactional records and model outputs.
Frontend	Lender Risk Dashboard, Borrower Dashboard, Risk Alerts, Health Score Viz	Role-based UIs for lenders and borrowers surfacing model outputs.

Model Dependency Chain

The five ML models form a dependency chain: raw transactions feed the Health Score and Anomaly models; the Health Score feeds Default Risk; risk outputs and spending trends feed the Wellness engine. Cash-Flow Forecasting operates independently but its low-balance flag feeds into Wellness.

Model	Depends On	Algorithm	Output
Financial Health Score	Bank transactions, profile data	XGBoost Regression	Score 0-100 + risk label
Loan Default Risk	Health Score, missed EMIs, income variance	XGBoost Classifier	Probability + Safe / Watch / High
Transaction Anomaly	Txn amount, time, merchant, balance	Isolation Forest	Anomaly score + boolean flag
Cash-Flow Forecast	Balance time-series, salary cycles, EMI dates	ARIMA / LSTM	7 / 30-day balance + low-balance flag
Wellness Recommendation	Savings rate, debt ratio, risk level, spending	Decision Tree / Rule Engine	Actionable advice categories

03 — TECHNOLOGY STACK & DEPENDENCIES

Technology Stack & Dependencies

Every major library, framework, and service used in FinPulse is listed below, grouped by layer, with version targets and the rationale for selection.

Data Ingestion & Processing

Package / Tool	Version	Purpose
Apache Airflow	2.8+	Pipeline / DAG orchestration for nightly ETL jobs
Apache Kafka	3.6+	Real-time transaction event streaming
Apache Spark (PySpark)	3.5+	Distributed batch feature computation
Pandas	2.1+	In-memory data manipulation and prototyping
SQLAlchemy	2.0+	ORM for PostgreSQL access within Python services

Feature Store

Package / Tool	Version	Purpose
Feast	0.38+	Open-source feature store (offline + online)
Redis	7.2+	In-memory online feature store backend
PostgreSQL	15+	Offline feature store and primary transactional DB
Parquet / S3	—	Columnar offline storage for historical features

ML Modelling

Package / Tool	Version	Purpose
XGBoost	2.0+	Health Score regression and Default Risk classification
LightGBM	4.1+	Faster boosting alternative; used for A/B comparisons
scikit-learn	1.4+	Preprocessing, baseline models, Isolation Forest
statsmodels / pmdarima	0.14+	ARIMA cash-flow forecasting
PyTorch / LSTM	2.1+	Advanced sequence model for cash-flow (v2)
SHAP	0.44+	Model explainability for credit decisions

MLOps & Registry

Package / Tool	Version	Purpose
MLflow	2.10+	Experiment tracking, model registry, artifact store
Prefect	2.16+	Lightweight pipeline orchestration (alternative to Airflow)
DVC	3.0+	Data version control for training datasets
Evidently AI	0.4+	Data drift and model performance monitoring

Model Serving & API

Package / Tool	Version	Purpose
FastAPI	0.110+	High-performance async REST API for ML microservices
Django + DRF	5.0+	Application-layer API with auth and reports
Docker	25+	Container runtime for each model microservice
Kubernetes (K8s)	1.29+	Container orchestration and autoscaling
NGINX	1.25+	Reverse proxy and load balancer
Celery + Redis	5.3+	Async task queue for batch prediction jobs

Observability

Package / Tool	Version	Purpose
Prometheus	2.50+	Metrics collection from all services
Grafana	10.2+	Dashboards for latency, throughput, drift alerts
OpenTelemetry	1.23+	Distributed tracing across microservices
Sentry	—	Error tracking and alerting

Frontend

Package / Tool	Version	Purpose
React 18 + TypeScript	18 / 5.4	Component-based UI for both dashboards
Recharts / D3.js	2.10+	Data visualisation (health score, risk charts)
React Query (TanStack)	5+	Server-state management and API caching
Tailwind CSS	3.4+	Utility-first styling

Infrastructure & DevOps

Package / Tool	Version	Purpose
AWS / GCP	—	Cloud hosting (EC2/GKE, S3/GCS, RDS)
Terraform	1.7+	Infrastructure-as-code provisioning
GitHub Actions	—	CI/CD pipelines for test, build, deploy
pytest + Great Expectations	—	Unit tests and data quality assertions

04 — PHASE 1: INFRASTRUCTURE & DATA LAYER SETUP

Phase 1 — Infrastructure & Data Layer Setup

Duration: Weeks 1-3 | **Team:** DevOps, Data Engineering, Backend

Phase 1 establishes the foundational infrastructure that all subsequent phases depend on. No ML work begins until data ingestion is stable and the database schema is validated.

Task	Owner	Dependency	Output
Provision cloud environment (VPC, IAM, S3/GCS buckets)	DevOps	Cloud account approved	Cloud infra ready
Spin up PostgreSQL RDS instance with schema migrations	Backend	Cloud infra	DB schema v1 live
Set up Apache Kafka cluster for transaction streaming	Data Eng.	Cloud infra	Kafka topics live
Build Data Ingestion API (Banking APIs + Credit Bureaus)	Backend	DB schema, Kafka	Ingestion API v1
Implement ETL: Cleaning & Preprocessing pipeline	Data Eng.	Ingestion API	Clean data in DB
Configure Airflow DAGs for nightly batch ETL	Data Eng.	ETL pipeline	Automated ETL runs
Set up Feast feature store (offline: Parquet/S3, online: Redis)	Data Eng.	ETL pipeline	Feature store scaffold
Deploy monitoring: Prometheus + Grafana base stack	DevOps	Cloud infra	Observability live

Key Dependencies & Risks

- ⚠ **Banking API access:** Third-party Banking API credentials must be obtained before the Ingestion API can be built. Allow 1-2 weeks for approval.
- ⚠ **Credit Bureau integration:** Bureau data enrichment requires signed data-sharing agreements. Start legal review in Week 1.
- ⚠ **Schema finalisation:** The PostgreSQL schema must be agreed upon before Feast feature definitions are written.

05 — PHASE 2: ML FEATURE ENGINEERING PIPELINE

Phase 2 — ML Feature Engineering Pipeline

Duration: Weeks 3-6 | **Team:** Data Engineering, ML Engineering

Phase 2 transforms clean, ingested data into model-ready features. The feature store becomes the single source of truth for both offline training and real-time inference, preventing training-serving skew.

Feature Groups & Computation

Feature Group	Source Tables	Computation	Store Type
Financial Features	transactions, accounts	Monthly income/expense totals, savings rate, EMI/Income ratio, cashflow volatility	Offline + Online
Transaction Behaviour	transactions	Spend-by-category ratios, frequency distribution, recency patterns	Offline + Online
User Metrics	user_profiles, loans	Credit history length, active loan count, debt-to-income ratio, missed EMI count	Offline + Online
Balance Time-Series	accounts (daily snapshots)	Rolling 7 / 30 / 90-day balance, balance trend slope, min-balance flags	Offline (batch)

Pipeline Architecture

Batch jobs run nightly via Airflow, computing features over the preceding 24-hour window and materialising them to the offline Parquet store. A streaming Kafka consumer continuously updates the online Redis store with the latest account balance and recent transaction signals to support sub-second real-time scoring.

Phase 2 Tasks

Task	Owner	Dependency	Output
Define Feast feature views & entities for all feature groups	Data Eng.	Feature store scaffold	Feature definitions committed
Write PySpark aggregation jobs (income, expense, EMI, volatility)	Data Eng.	Clean data in DB	Offline feature tables
Build streaming consumer: Kafka → Redis online store update	Data Eng.	Kafka topics, Redis	Real-time feature updates
Implement Great Expectations data quality checks on features	Data Eng.	Feature tables	Quality validation suite
Materialise first historical feature snapshot for model training	Data Eng.	All feature jobs	Training dataset v1
Document feature definitions in data catalogue	Data Eng.	Feature views defined	Feature catalogue published

06 — PHASE 3: MODEL DEVELOPMENT & TRAINING

Phase 3 — Model Development & Training

Duration: Weeks 5-10 | *Team:* ML Engineering

Phase 3 covers the full ML lifecycle for all five models: data assembly, experimentation, hyperparameter tuning, evaluation, and registration in MLflow. Models are promoted to staging only after meeting agreed KPI thresholds.

Financial Health Score

Algorithm	XGBoost Regressor (baseline: Linear Regression)
Inputs	Monthly income/expense, savings rate, EMI/Income ratio, cashflow volatility, credit history length
KPIs	RMSE < 5, R² > 0.85
Notes	Train on 12+ months of historical data. Use 5-fold cross-validation. Apply SHAP for feature importance.

Loan Default Risk

Algorithm	XGBoost Classifier (baseline: Logistic Regression)
Inputs	Health Score, missed EMI count, income variance, expense spikes, active loan count
KPIs	AUC-ROC > 0.90, KS statistic > 0.40
Notes	Handle class imbalance with SMOTE. Calibrate probabilities using Platt scaling.

Transaction Anomaly Detection

Algorithm	Isolation Forest (alternative: One-Class SVM)
Inputs	Transaction amount, time-of-day, merchant category, post-transaction balance, frequency patterns
KPIs	Precision > 0.85 @ 5% FPR on labelled holdout
Notes	Unsupervised — no ground-truth labels required for training. Validate on a manually labelled fraud set.

Cash-Flow Forecasting

Algorithm	ARIMA (production), LSTM (v2)
Inputs	Daily balance time-series, salary deposit dates, recurring expense dates, EMI payment schedule
KPIs	MAPE < 8% over 7-day horizon, < 15% over 30-day
Notes	Use pmdarima for auto-ARIMA. LSTM requires GPU; train offline and export to ONNX for serving.

Wellness Recommendation

Algorithm	Decision Tree / Rule Engine (max depth ≤ 5)
Inputs	Savings rate, category spend ratios, debt-income ratio, Health Score, risk class
KPIs	Coverage > 95% of user states; explanation clarity audit
Notes	Keep depth shallow for full explainability. Validate rules with financial advisors before deployment.

07 — PHASE 4: MODEL SERVING & API LAYER

Phase 4 — Model Serving & API Layer

Duration: Weeks 9–13 | **Team:** ML Engineering, Backend, DevOps

Each trained model is packaged as an independent FastAPI microservice, containerised with Docker, and deployed to Kubernetes. The Model Registry (MLflow) gates promotions from staging to production. All endpoints are versioned and documented via OpenAPI.

API Endpoint Contracts

Endpoint	Method	Key Request Fields	Response Fields
/predict/health_score	POST	user_id, monthly_income, total_expense, savings_rate, emi_income_ratio, cashflow_volatility	health_score (float), risk_label (str)
/predict/default_risk	POST	user_id, health_score, missed_emi, income_variance, active_loans	default_probability (float), risk_class (str)
/detect/anomaly	POST	user_id, transaction: {amount, time, merchant_category}	anomaly_score (float), is_anomalous (bool)
/forecast/balance	POST	user_id, date, balance_history (list of floats)	forecast_7d, forecast_30d (floats), low_balance_risk (bool)
/recommend/wellness	POST	user_id, savings_rate, debt_income_ratio, risk_class	advice (str), recommendation_code (str)

Serving Infrastructure

Component	Technology	Role
ML Microservices (×5)	FastAPI + Uvicorn in Docker	One container per model; independent scaling and versioning
Prediction API Gateway	NGINX / Kong	Routes requests, rate-limits, and load-balances across model pods
Model Registry	MLflow Tracking Server	Version-controls model artefacts; gates staging → production promotions
Monitoring	Evidently AI + Prometheus	Data drift detection, latency percentiles, accuracy decay alerts
Async Batch Scoring	Celery + Redis queue	Nightly bulk scoring of all borrowers; results cached in PostgreSQL
Service Mesh	Kubernetes + HPA	Auto-scales pods under load; health-checks restart failed containers

08 — PHASE 5: APPLICATION & FRONTEND BUILD

Phase 5 — Application & Frontend Build

Duration: Weeks 10-15 | *Team:* Backend, Frontend

Django Application Layer

The Django + Django REST Framework application sits between the ML microservices and the frontend. It handles authentication (JWT), authorisation (RBAC for lenders vs borrowers), aggregates model outputs into composite reports, and writes final predictions to PostgreSQL.

Django Module	Responsibility	Key Libraries
users/	Registration, JWT auth, RBAC roles (lender / borrower / admin)	djangorestframework-simplejwt, dj-rest-auth
scores/	Fetch Health Score & Default Risk from ML services; cache in DB	httpx, celery
transactions/	Stream transaction events; trigger anomaly detection in real time	kafka-python, django-channels (WebSocket)
forecasts/	Serve cash-flow predictions; expose 7 / 30-day endpoints	httpx, pandas
recommendations /	Fetch wellness advice; log user interactions	httpx
reports/	Generate downloadable PDF / CSV financial reports	reportlab, pandas
admin/	Internal Django admin for model management and user operations	Django admin (built-in)

Frontend Architecture

The React 18 + TypeScript SPA is split into two role-gated sub-applications: the **Lender Risk Dashboard** and the **Borrower Dashboard**, each accessible via JWT-protected routes.

Screen / Component	Dashboard	API Endpoint	Visualisation
Health Score Viz	Borrower	/scores/health_score/	Radial gauge (D3.js)
Risk Alert Feed	Borrower + Lender	/transactions/anomalies/	Real-time list (WebSocket)
Cash-Flow Chart	Borrower	/forecasts/balance/	Area chart (Recharts)
Wellness Recommendations	Borrower	/recommendations/	Card list with action CTAs
Borrower Risk Table	Lender	/scores/default_risk/	Sortable data grid
Portfolio Health Overview	Lender	/reports/portfolio/	Bar + heatmap (Recharts)
Financial Reports Download	Lender + Borrower	/reports/	PDF / CSV export button

09 — PHASE 6: MLOPS, MONITORING & CI/CD

Phase 6 — MLOps, Monitoring & CI/CD

Duration: Weeks 12-16 (ongoing) | *Team:* ML Engineering, DevOps

CI/CD Pipeline

Stage	Trigger	Steps	Gate
PR Validation	Pull request opened	Lint, unit tests (pytest), data quality checks	All tests green
Build & Push	Merge to main	Docker build, push to container registry, tag with git SHA	Image scan clean
Staging Deploy	Image push	Helm upgrade to staging K8s namespace; smoke tests	Smoke tests pass
Model Evaluation	Weekly schedule	Re-run evaluation on latest data; compare to baseline KPIs	KPIs met
Production Deploy	Manual approval	Blue/green deploy; traffic shift 10% → 50% → 100%	Error rate < 0.1%
Rollback	Automated (on alert)	Helm rollback to previous release; alert on-call engineer	< 5 min TTR

Monitoring & Alerting Strategy

Metric	Tool	Alert Threshold	Action
API latency (p99)	Prometheus + Grafana	> 500 ms	Scale up pod replicas
Model accuracy drift	Evidently AI	AUC drops > 5% vs baseline	Trigger retraining DAG
Feature distribution shift	Evidently AI	PSI > 0.25 on key features	Investigate data pipeline
Anomaly false-positive rate	Custom metric	> 10% FPR on sampled transactions	Review Isolation Forest threshold
Error rate (5xx)	Prometheus	> 0.5% of requests	Page on-call engineer
Cash-flow forecast MAPE	MLflow	> 15% on 7-day horizon	Retrain ARIMA / switch to LSTM

Retraining Schedule

Model	Frequency	Trigger	Validation Before Promote
Financial Health Score	Monthly	Scheduled + drift alert	RMSE < 5, R ² > 0.85
Loan Default Risk	Monthly	Scheduled + drift alert	AUC > 0.90
Transaction Anomaly	Quarterly	Drift alert (PSI > 0.25)	Precision > 0.85
Cash-Flow Forecast	Weekly	Scheduled (rolling window)	MAPE < 8% (7-day)
Wellness Recommendation	Quarterly	Rule review by domain experts	Coverage > 95%

10 — END-TO-END DATA FLOW

End-to-End Data Flow

Data moves from external sources through every layer of FinPulse to the user-facing dashboards. Two parallel paths exist: a real-time scoring path (< 200 ms) and a nightly batch path.

① External Services	Credit Bureaus · Identity Verification · Banking APIs	Enrich data →
② Data Sources	Bank Transactions · User Profile Data · Loan Information Spending Categories · Account Balance History	Ingest →
③ Data Processing	Data Ingestion API → Cleaning & Preprocessing → Feature Engineering → Aggregation	Features →
④ Feature Store	Offline Store (Parquet/S3) ← Spark batch jobs Online Store (Redis) ← Kafka stream consumer	Fetch →
⑤ ML Models	Health Score Default Risk Anomaly Detection Cash-Flow Forecast Wellness Recommendation	Predict →
⑥ Model Serving	Prediction APIs (FastAPI) · ML Microservice Model Registry (MLflow) · Monitoring (Evidently)	Serve →
⑦ Application Layer	Django REST API · JWT Auth · Financial Reports Celery async tasks · PostgreSQL persistence	Display →
⑧ Database Layer	PostgreSQL — Transaction Data · Model Predictions · User Data	
⑨ Frontend	Lender Risk Dashboard · Borrower Dashboard Risk Alerts · Health Score Visualisation · Cash-Flow Chart	

Real-Time vs Batch Scoring Paths

Path	Trigger	Latency Target	Models Involved
Real-time scoring	User opens dashboard / new transaction arrives	< 200 ms	Health Score, Default Risk, Anomaly
Streaming anomaly	Kafka transaction event	< 500 ms	Anomaly Detection
Nightly batch	Airflow DAG at 02:00 UTC	Complete by 06:00 UTC	All 5 models (bulk scoring)
Weekly forecast	Airflow DAG (Monday)	Complete by 04:00 UTC	Cash-Flow Forecast
On-demand report	User requests PDF export	< 10 s	Aggregated from DB cache

11 — RISK REGISTER & MITIGATION

Risk Register & Mitigation

Risks identified across all project phases. Each is rated by Likelihood (L) and Impact (I) on a 1–3 scale. Score = L × I. High scores require immediate mitigation.

Risk	Phase	L	I	Score	Mitigation Strategy
Banking API delays / access denied	1	2	3	6	Begin API credential applications in Week 0; build a mock API for parallel development.
Training-serving skew (feature inconsistency)	2–3	2	3	6	Use Feast for both training and serving; enforce Great Expectations data quality checks.
Insufficient labelled fraud data	3	3	2	6	Use unsupervised Isolation Forest; supplement with synthetic data generation.
Model accuracy below KPI thresholds	3	2	3	6	Early baseline models; A/B test XGBoost vs LightGBM; iterative feature engineering.
High inference latency (> 500 ms)	4	2	2	4	Profile bottlenecks; cache frequent predictions; autoscale Kubernetes pods.
Data privacy / GDPR compliance breach	1–5	1	3	3	PII masking at ingestion; encrypt at rest and in transit; conduct DPIA in Phase 1.
Frontend performance on large datasets	5	2	2	4	Paginated API responses; virtual scrolling; React Query caching.
Model drift in production	6	2	3	6	Evidently AI monitoring; automated retraining triggers; weekly KPI reviews.
CI/CD pipeline failures blocking deploy	6	1	2	2	Blue/green deployments; automated rollback on error-rate spike.
Credit bureau data quality issues	1–2	2	2	4	Data quality assertions at ingestion; fallback to internal data if bureau feed fails.

Score 6 (High)	Requires immediate mitigation plan and weekly tracking by project lead.
Score 4 (Medium)	Mitigation plan in place; monitor fortnightly.
Score ≤ 3 (Low)	Acceptable residual risk; log and review monthly.

12 — TIMELINE & MILESTONE SUMMARY

Timeline & Milestone Summary

FinPulse spans approximately 16 weeks from kickoff to production launch. Phases overlap to maintain momentum. The critical path runs through Phases 1-4.

We ek	Phase	Milestone / Deliverable	Owner
1	Phase 1	Cloud infra provisioned; Kafka cluster live	DevOps
2	Phase 1	PostgreSQL schema v1; Data Ingestion API v1	Backend
3	Phase 1	Nightly ETL DAG running; monitoring stack live	Data Eng.
4	Phase 2	Feast feature store scaffold; feature views committed	Data Eng.
5	Phase 2	Offline PySpark feature jobs running	Data Eng.
6	Phase 2	Training dataset v1 materialised; GE checks passing	Data Eng.
7	Phase 3	Health Score model v1 trained; KPIs validated	ML Eng.
8	Phase 3	Default Risk model trained; Anomaly model baselined	ML Eng.
9	Phase 3	Cash-Flow ARIMA trained; Wellness rules defined	ML Eng.
10	Phase 3-4	All models registered in MLflow; staging deploy	ML Eng. + DevOps
11	Phase 4	All 5 FastAPI microservices live on staging Kubernetes	ML Eng. + DevOps
12	Phase 4-5	Django API v1 (auth, scores, transactions) complete	Backend
13	Phase 5	React dashboards (Lender + Borrower) in beta	Frontend
14	Phase 5-6	Full integration test; CI/CD pipeline green	All
15	Phase 6	Load testing; security audit; DPIA sign-off	DevOps + Legal
16	Launch	Production launch; monitoring + on-call runbook live	All

Team Structure

Role	Count	Primary Phases	Key Responsibilities
DevOps / Platform Engineer	1-2	1, 4, 6	Cloud infra, Kubernetes, CI/CD, monitoring
Data Engineer	2	1, 2	ETL, Kafka, Spark, Feast feature store
ML Engineer	2-3	2, 3, 4, 6	Model training, serving, MLOps, retraining pipelines
Backend Engineer	2	1, 4, 5	Django API, authentication, async tasks, DB management
Frontend Engineer	1-2	5	React dashboards, D3.js charts, UX
Product / QA	1	All	Requirements, testing, stakeholder communications

APPENDIX — QUICK-REFERENCE BUILD CHECKLIST

Quick-Reference Build Checklist

Use this checklist to track readiness before each phase begins. All items in a phase must be confirmed before proceeding to the next.

PHASE 1 — INFRASTRUCTURE

- ☐ Cloud accounts and billing set up with cost alerts configured
- ☐ VPC, subnets, IAM roles, and S3/GCS buckets provisioned via Terraform
- ☐ PostgreSQL RDS instance running with connection pool configured
- ☐ Apache Kafka cluster live with transaction and user-event topics created
- ☐ Data Ingestion API connecting to all Banking APIs and Credit Bureau feeds
- ☐ Airflow running with at least one healthy nightly ETL DAG
- ☐ Prometheus + Grafana base monitoring stack operational

PHASE 2 — FEATURE ENGINEERING

- ☐ Feast feature store configured with offline (Parquet/S3) and online (Redis) backends
- ☐ All five feature groups defined and committed to feature repository
- ☐ PySpark aggregation jobs producing income, expense, EMI, and volatility features
- ☐ Kafka consumer updating Redis online store with sub-second latency
- ☐ Great Expectations data quality suite with ≥ 20 assertions passing
- ☐ Training dataset v1 materialised and documented in data catalogue

PHASE 3 — MODEL TRAINING

- ☐ Health Score model: RMSE < 5 , $R^2 > 0.85$ on holdout set
- ☐ Default Risk model: AUC-ROC > 0.90 , KS > 0.40 on holdout set
- ☐ Anomaly Detection model: Precision > 0.85 at 5% FPR
- ☐ Cash-Flow Forecast: MAPE $< 8\%$ (7-day), $< 15\%$ (30-day)
- ☐ Wellness Recommendation: coverage $> 95\%$ of user states
- ☐ All models registered in MLflow with metadata and artefacts
- ☐ SHAP explanations generated for all credit-facing models

PHASE 4 — MODEL SERVING

- ☐ All 5 FastAPI microservices containerised and pushed to registry
- ☐ Kubernetes deployments with Horizontal Pod Autoscaler configured for each service
- ☐ All API endpoints returning correct JSON with p99 latency < 200 ms
- ☐ Model Registry promotion gates enforced (staging → production)
- ☐ Evidently AI drift monitoring active for all deployed models

PHASE 5 — APPLICATION & FRONTEND

- ☐ Django API: auth, scores, transactions, forecasts, recommendations, reports modules live
- ☐ JWT authentication and RBAC (lender / borrower) enforced on all endpoints
- ☐ Lender Risk Dashboard: portfolio view, risk table, anomaly feed operational
- ☐ Borrower Dashboard: health score gauge, cash-flow chart, wellness cards live
- ☐ PDF / CSV report export functional and tested

PHASE 6 — MLOPS & LAUNCH

- ☐ GitHub Actions CI/CD pipeline green end-to-end
- ☐ Blue/green deployment tested with successful traffic switch
- ☐ Load test: 1,000 concurrent users, < 500 ms p99 on all endpoints
- ☐ Security audit and DPIA completed with sign-off obtained
- ☐ On-call runbook documented and engineering team trained
- ☐ Production monitoring dashboards live with alert routing configured